

Kapish Shukla

Kunal_SemVI-ProjectDocumentation.docx

 TY PROJECT Odd Sem TY Projects 2025 Vidyalkar School of Information Technology

Document Details

Submission ID

trn:oid::1:3505932819

Submission Date

Mar 13, 2026, 6:24 PM GMT+5:30

Download Date

Mar 13, 2026, 6:50 PM GMT+5:30

File Name

Kunal_SemVI-ProjectDocumentation.docx

File Size

990.1 KB

44 Pages

9,312 Words

46,506 Characters

16% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Exclusions

- 12 Excluded Matches

Match Groups

-  **43 Not Cited or Quoted 16%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 2%  Internet sources
- 3%  Publications
- 15%  Submitted works (Student Papers)

Match Groups

- 43 Not Cited or Quoted 16%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 2% Internet sources
- 3% Publications
- 15% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers	Vidyalankar Institute of Technology	14%
2	Internet	digital.lib.usu.edu	<1%
3	Student papers	University of Southern California	<1%
4	Publication	B. G. Nagaraja, S. Kannadhasan. "Information and Communication Systems", CRC...	<1%
5	Publication	Paolo Ferro, Harinadh Vemanaboina, Chander Prakash. "Computational Techniqu...	<1%
6	Internet	www.jetir.org	<1%
7	Publication	Ahmed A. Abd El-Latif, Mohammed A ElAffendi, Mohamed Ali AlShara, Yassine Ma...	<1%
8	Internet	www.mdpi.com	<1%
9	Publication	Hanaa ZainEldin, Samah A. Gamel, Fatma M. Talaat, Mansourah Aljohani et al. "Sil...	<1%
10	Publication	T. Ananth Kumar, R. Rajmohan, M. Niranjnamurthy, G. Sambasivam. "Deep Lear...	<1%

AMERICAN HAND SIGN LANGUAGE CLASSIFICATION

A Project Report

Submitted in partial fulfilment of the
requirements for the award of the Degree of

BACHELOR OF SCIENCE (DATA SCIENCE)

By

Kapish Shukla

Kunal Solanki

Seat Number: _____

Under the esteemed guidance of

Mr. Shajil Kumar P A

Assistant Professor, Department of Information Technology

And Data Science



DEPARTMENT OF INFORMATION TECHNOLOGY AND DATA SCIENCE

VIDYALANKAR SCHOOL OF INFORMATION TECHNOLOGY

(Affiliated to University of Mumbai)

MUMBAI, 400 037

MAHARASHTRA

2024 - 2025

VIDYALANKAR SCHOOL OF INFORMATION TECHNOLOGY**(Affiliated to University of Mumbai)****MUMBAI-MAHARASHTRA-400037****DEPARTMENT OF INFORMATION TECHNOLOGY****AND DATA SCIENCE****CERTIFICATE**

This is to certify that the project entitled, "**American Hand Sign Language Classification**",
is bonafied work of **KAPISH SHUKLA AND KUNAL SOLANKI** bearing Seat No:
_____ submitted in partial fulfilment of the requirements for the award of
degree of BACHELOR OF SCIENCE in DATA SCIENCE from University of Mumbai.

Internal Guide**Coordinator****Internal Examiner****External Examiner****Date:****College Seal****Principal**

ABSTRACT

Deaf and mute people face many problems when they try to talk to others because most people don't know American Sign Language (ASL). This makes it very hard for them to share their feelings or needs in daily life. So, we wanted to make a smart system that can understand hand signs and tell everyone what they mean. This way, technology can help bridge the gap between different people.

For this project, we collected thousands of pictures of hand gestures for letters like A, B, C, D, and E from different places. We used Python and special tools like TensorFlow to train a "brain" for the computer called a CNN model. Before training, we made all images the same size and cleaned them so the computer could learn better. This helps the computer recognize the hand shapes very quickly.

Our results show that the computer is now very good at identifying these five sign language letters correctly. We checked the accuracy of our model, and it performs well even with different types of hand photos. It feels great to see that a machine can understand what a person is saying through gestures. This proves that our idea works and can be very useful for society.

In the future, we want to add all the letters of the alphabet and even full sentences to our system. We also plan to make a mobile app so people can use it anywhere on their phones. Our main goal is to make a real-time system that works with live video and cameras. We hope this project helps many people live a better and easier life.

ACKNOWLEDGEMENT

I take this opportunity to thank all those who helped me during my semester project. Otherwise, it would have been impossible without their great help and advice.

I should thank Vidyalankar School of Information Technology for offering me this opportunity to do the work and for establishing an amiable learning atmosphere that promotes newness and research. I will particularly enjoy the resources and support I will receive from the institution during the period of my project.

First and foremost, I would like to put on record the special gratitude I have in my heart for my project guide, Prof. Shajil Kumar; without all his advice, encouragement, and feedback during the project, much of this couldn't have been possible. He has been an inspiring mentor and an endless source of inspiration. All the expertise and knowledge in the fields of e-commerce and machine learning have been crucial for me to face challenges and enhance the quality of work.

I would like to extend my gratitude to all faculty members of Vidyalankar School of Information Technology. Their teaching and guidance have contributed significantly to my learning. Special thanks to the administrative staff for their assistance and to fellow students for valuable discussion and support throughout this journey.

DECLARATION

I hereby declare that the project entitled, “**AMERICAN HAND SIGN LANGUAGE CLASSIFICATION**” done at Vidyalankar School of Information Technology, has not been in any case duplicated to submit to any other universities for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfilment of the requirements for the award of degree of **BACHELOR OF SCIENCE (DATA SCIENCE)** to be submitted as final semester project as part of our curriculum.

Name and Signature of the Student

TABLE OF CONTENTS

Chapter 1 Introduction.....	9
1.1 Background	10
1.2 Objectives	14
1.3 Purpose, Scope, Applicability (Feasibility Study).....	15
Chapter 2 Dataset Description.....	17
2.1 Introduction.....	17
2.2 Data Source.....	17
2.3 Dataset Characteristics.....	17
2.4 Data Features	18
2.5 Dataset Usage	19
2.6 Limitations of the Dataset	19
Chapter 3 Methods and Algorithms.....	20
3.1 Overview of Methods Used	20
3.2 Models Used.....	20
3.3 Feature Selection Methods.....	21
3.4 Model Evaluation.....	21
3.5 Hyperparameter Tuning	22
3.6 Challenges and Future Directions	22
Chapter 4 Project Analysis	23
4.1 Data Mapping and Folder Structure.....	23
4.2 Data Preprocessing and Feature Engineering for Image Classification	27
4.3 Data Distribution and Feature Analysis	29
4.4 Model Performance	30
4.5 Hyperparameter Tuning	32
4.6 Impact of Increasing the Number of Rāgas.....	33
4.7 Error Analysis	34
4.8 Challenges and Limitations.....	35
Chapter 5 Final Results.....	36
5.1 Model Performance Evaluation.....	36
5.2 Model Limitations and Challenges.....	37
Chapter 6 Conclusion and Future Scope	38
6.1 Conclusion	38
6.2 Future Scope.....	38

6.3 Closing Remarks	39
References	39
Bibliography	40
Glossary	41
Summary	42
Further Reading	43
Plagiarism Report	44

List of Figures

Figure 1: The CSV structure containing image files path for each Hand Sign.....	24
Figure 2: Code for recording existing image file location into a Python list.....	25
Figure 3: Code for concatenating pandas dataframes for file management.....	25
Figure 4: Resetting the index and mapping the categorical labels to numbers.....	25
Figure 5: Structured dataset for feature files processing	26
Figure 6: Code for Data Augmentation of Image file.....	29
Figure 7: Distribution of Hand Sign Names	30
Figure 8: The accuracy of each epoch	31
Figure 9: Evaluation report on randomly blurred images.....	32
Figure 10: Testing on training data.....	32
Figure 11: Training results with callbacks implemented.....	33
Figure 12: Model Structure.....	34
Figure 13: Misclassifications caused by feature overlap	35

Chapter 1 Introduction

Communication is very important for everyone to talk and share ideas. But for people who cannot speak or hear, it is very difficult because they use American Sign Language (ASL). Most normal people don't understand these hand signs, so there is a big gap in communication. We want to solve this problem using technology so everyone can understand them easily.

In our project, we are using Computer Vision and Deep Learning to identify hand gestures. The main goal is to make a system that can look at a hand photo and tell which letter it is. This is helpful for making a bridge between deaf people and those who don't know sign language. It makes the world a better place where everyone can talk to each other.

We are using a special computer program called a Convolutional Neural Network (CNN) for this task. This program works like a human brain and learns how different hands look for different letters. We have used many pictures from the internet to teach our model. By doing this, the computer becomes very smart at recognizing signs even if the light or background is different.

Nowadays, artificial intelligence is helping in many areas like health and education. Our project is also a small step to help society using these new technologies. In the future, this can be turned into a real app that works on any smartphone. We believe that such projects are very important for helping people and making their life easier.

1.1 Background

1.1.1 Overview

Our project is all about helping people who cannot speak and use hand signs to talk. We have built a smart computer system that can see pictures of American Sign Language and tell us what letter it is. Right now, our model can identify five letters which are A, B, C, D, and E very accurately. We used a lot of photos from Kaggle to teach the computer how different hands look in different lights. It is like giving eyes to the computer so it can understand the silent language of deaf people.

To make this work, we used Python language and a very cool technology called CNN, which is part of Deep Learning. First, we collected thousands of images and cleaned them by resizing and making them look clear for the machine. Then, we trained our model so it can learn the patterns of each hand gesture perfectly. Our goal is to make sure the computer doesn't make mistakes when it sees a new hand sign. In the future, we want to make this into a real app that can help everyone communicate without any problems.

1.1.2 Rationale for choosing this topic

I chose this topic because I saw that many people who cannot speak or hear face big problems in their daily life. They use hand signs to talk, but most of us don't understand what they are saying. I felt very sad about this communication gap and wanted to do something to help. By using technology, we can make a bridge so that everyone can understand their feelings and needs easily.

Another reason is my interest in how computers can see and understand things like humans. I wanted to learn how Artificial Intelligence and Python can solve real-world problems. This project is a perfect way to use my coding skills for a good cause. It makes me happy to think that my small project can one day help a person communicate with the whole world.

1.1.3 Significance

The significance of this project is very big because it helps people who cannot talk normally. Many times, deaf people feel alone because nobody understands their hand signs in shops or hospitals. Our system can quickly translate these signs into text so everyone can understand them easily. This will give them more confidence and make them feel like a part of our society.

Another important thing is that this project uses new technology like AI to do something good for humans. It shows that computers are not just for games but can also help in solving real problems. In the future, this can be used in schools to teach sign language to other kids too. It makes communication fast and easy for everyone without needing a human translator all the time.

1.1.4 Challenges

Working on this project was not easy because there were many challenges. The biggest problem was the background and lighting in the photos. Sometimes the computer got confused when the room was dark or if there were other things behind the hand. We had to spend a lot of time cleaning the images so the machine could focus only on the hand signs and not the background.

Another challenge was that some hand signs look very similar to each other. For example, letters like 'A' and 'E' can look almost the same if the camera angle is not perfect. It was hard to train the CNN model to see the small differences between these gestures. Also, collecting so many thousands of pictures and processing them took a lot of time and computer power.

1.1.5 Justification

The justification for this project is that it uses technology to solve a real human problem. Even though there are many apps, there is still no easy way for deaf people to talk to everyone in real-time. By making this system, we are proving that simple cameras and AI can work together to understand sign language. It is a cheap and effective way to help people who cannot speak, making their life much better.

Also, we used a very strong model called CNN which is perfect for identifying images. This ensures that our project is not just a demo but a working solution that can be improved further. Using Python and open-source data makes it easy for others to learn and grow this idea. We believe that such projects are the best way to use science for helping the community and bringing people closer.

1.1.6 Historical Methods

Overview of Historical Sign Recognition -

In the old days, there was no technology to help deaf people talk to others easily. People tried to write down sign language or use books to teach gestures, but it was very slow and difficult. There was no way for a computer to "see" or "understand" these signs automatically. Everything was done manually, and because of this, the communication gap between people remained very large for a long time.

Later, scientists started using simple cameras and basic computer rules to recognize shapes, but it didn't work very well in dark or messy rooms. Now, because of new technology like Deep Learning and CNN, computers have become much smarter. They can learn from thousands of photos and identify hand signs almost like a human eye. This history shows how we moved from simple books to smart AI systems to help everyone communicate.

Challenges of Historical Methods

The old methods for recognizing sign language had many big problems. Earlier, computers were not smart enough to understand hand shapes from a simple photo. People used to wear special electronic gloves with many wires to track hand movements, but these gloves were very expensive and uncomfortable to wear. It was very difficult for a normal person to use such heavy machines just to say a simple "Hello" or "Thank you."

Another challenge was that the old computer rules only worked if the background was plain and the light was perfect. If the person moved a little bit or if the room was a bit dark, the system would fail completely. Also, those systems could only understand a few signs and were very slow to give results. This is why those historical methods were not practical for real life, and that's why we need modern AI today.

1.1.7 Modern Advancements

Innovations in Technology

Technology has changed a lot and now we have amazing things like Artificial Intelligence and Deep Learning. In the past, computers could only follow simple commands, but now they can "see" and "think" like humans using CNN models. This innovation is great because it allows our project to recognize hand gestures from a simple camera without needing any extra wires or expensive tools. It makes everything much faster and smarter than before.

Another big innovation is that we can now store and process thousands of images in just a few minutes using cloud power. Before, it was impossible for a normal student to train such big models, but now it is very easy. These new technologies help us to build apps that can help disabled people in real-time. Because of these smart innovations, we can dream of a world where everyone can talk to each other without any language problems.

Advantages of Modern Techniques

Modern techniques like CNN and Deep Learning are very good because they are very accurate and fast. Unlike old methods, we don't need any special gloves or expensive sensors to understand the hand signs. We can just use a normal laptop camera or a mobile phone to see the gestures. This makes the system very easy to use for everyone and it works well even if the person is moving their hand a little bit.

Another big advantage is that these modern models can learn by themselves from thousands of examples. Once the model is trained, it can recognize signs in different types of rooms, whether it is bright or a bit dark. It can also handle different hand sizes and shapes very easily. This makes our project very strong and reliable for helping deaf people in the real world without any mistakes.

1.2 Objectives

Objective 1: Recognize and Classify Hand Signs

The main goal of our project is to use a computer to identify different hand signs from American Sign Language. We want to build a smart system that can look at a hand gesture and quickly tell if it is the letter A, B, C, D, or E. By using machine learning, we can make this process automatic so that people don't have to translate everything manually.

Objective 2: Help People Communicate Better

Another important objective is to help deaf and mute people talk to others without any fear. This project will work like a bridge between sign language and normal language so everyone can understand each other. Our system will show the text for each sign, making it very easy for students and common people to learn and use sign language in their daily life.

Objective 3: Use Smart Technology for Good

We want to use modern tools like Python and CNN to solve a real human problem and make the world better. This objective is about training a computer "brain" to see patterns in hand shapes just like humans do. By doing this, we are making a robust system that can work accurately in different places and help many people live an easier and more inclusive life.

1.3 Purpose, Scope, Applicability (Feasibility Study)

1.3.1 Purpose

The main purpose of our project is to create a smart tool that helps deaf and mute people communicate with everyone very easily. We want to use a simple camera and a computer to understand hand signs and turn them into text that anyone can read. This way, a person who doesn't know sign language can still understand what the other person is trying to say. Our goal is to use modern technology like AI to remove the communication gap and make sure no one feels left out in society.

1.3.2 Scope

The scope of this project covers the development of a machine learning-based hand sign classification system that can process images of American Sign Language. The system will:

- Extract relevant features like hand shapes, finger positions, and patterns from the images.
- Utilize smart CNN methods to analyze these visual features automatically.
- Classify the hand signs into distinct categories based on their shapes (like letters A, B, C, D, and E).
- Support up to 5 different alphabetic signs from the American Sign Language tradition.
- Offer insights into how computers recognize gestures, which can be used for helping deaf and mute people.

The project will be limited to analyzing static images of hand signs and will not address moving videos or full sign language sentences for now.

1.3.3 Applicability

This system is applicable in a variety of fields:

- Special Education: The system can be used by teachers and students in special schools to learn and practice American Sign Language (ASL) more easily.
- Healthcare and Hospitals: Doctors and nurses can use this tool to understand what a deaf or mute patient is saying, helping them give better treatment.

- **Social Inclusion:** This project helps in making public places like banks or parks more friendly for disabled people by providing an easy way to communicate.
- **Assistive Technology:** The project shows how we can use AI and cameras to help people with disabilities, and it can be turned into a mobile app for everyday use.

1.3.4 Significance of the System

The significance of our system is that it makes life much easier for people who cannot hear or speak. Usually, these people feel left out because most of us don't understand their sign language, but our project changes that by showing the meaning of their signs on a screen. It is like a 24-hour translator that helps them in shops, hospitals, or schools without any cost. This technology is very important because it uses smart AI to bring everyone together and gives deaf people the confidence to talk to the world.

1.3.5 Technical feasibility

Our project is technically feasible because we have all the right tools like Python and TensorFlow to make it work. Most modern computers and laptops have enough power to run the CNN models that we are using for identifying hand signs. Also, we don't need any expensive hardware; just a simple webcam is enough to capture the images. Since there are many datasets available on the internet for sign language, training the model has become much easier for us. All these things show that our project can be easily built and used in the real world without any big technical problems.

Chapter 2 Dataset Description

2.1 Introduction

The dataset is the most important part of our project because it is like a textbook for the computer. To teach our model how to recognize hand signs, we collected thousands of images of different hand gestures. Each image shows a specific letter from the American Sign Language like A, B, C, D, and E. Without these photos, the computer would not know what a hand looks like or how the fingers move for each sign.

We made sure to include many different types of photos in our dataset. This means we took pictures of different people's hands with different skin colors and sizes. We also used different backgrounds and lighting so that the computer becomes very smart. If the dataset has a lot of variety, our model will work better in the real world when someone uses it in a room or outside.

Before giving these photos to the computer, we had to organize them into different folders. Each folder has a label, so the computer knows which sign is which. This step is called data collection, and it is the first thing we did to start our project. By having a clean and big dataset, we made sure that our Sign Language system is very strong and gives the right results.

2.2 Data Source

To get the photos for our project, we used a very famous website called Kaggle. It is a place where many smart people share big collections of data for free. We downloaded the "Sign Language MNIST" dataset, which has thousands of pictures of hand gestures for different letters. This helped us a lot because we didn't have to click thousands of photos ourselves with our own camera.

We also used some photos that we took with our own laptop webcam to test the model. By using both Kaggle photos and our own photos, we made sure the computer can recognize signs from anyone. This mix of data makes our project more realistic and strong. It was very easy to download the files and put them into our Python code to start the training.

2.3 Dataset Characteristics

Our Hand Sign Language dataset consists of:

1. **Thousands of Images:** We have a very big collection of photos for each hand sign. This includes many different people showing signs like A, B, C, D, and E so the computer can learn them properly.
2. **Feature Data:** For every photo, we look at important things like where the fingers are and the shape of the palm. These "features" are very important because they help the computer tell the difference between one letter and another.

These photos are special because they were taken in different rooms with different lights. Some photos are close-up and some are from a distance, which helps the model become more powerful. Even though the original files were just simple images, we converted them into numbers (pixels) so that our Python code could understand them easily. This large variety of photos makes sure our system is very accurate and works for everyone.

2.4 Data Features

The dataset has many important features that help the computer recognize hand signs. The most important things we look for in each photo are:

- **Hand Shape:** This tells the computer the overall look of the hand, like if the fist is closed or if the palm is open.
- **Finger Positions:** This is very important because it tracks where each finger is pointing (up, down, or folded) to identify the letter.
- **Pixel Values:** This is the color and brightness of each tiny dot in the photo, which helps the computer see the hand against the background.
- **Key Points:** These are special dots on the knuckles and fingertips that the system uses to measure the distance between fingers.

These features are taken from the photos using special code that analyzes every part of the hand. The main goal is to capture the most important details so that even if the photo is a bit blurry, the computer can still guess the right sign. By looking at these things, the system understands the difference between an "A" sign and a "B" sign easily.

2.5 Dataset Usage

To use the dataset, we first divided all the photos into two groups. The biggest group is for "Training," where the computer looks at the photos and learns how the hand signs look. The second group is for "Testing," where we check the computer to see if it can identify new photos correctly. This is just like how we study from a book first and then give an exam to see if we learned everything.

We also used some special tricks to make the dataset even better. We rotated some photos and changed the brightness so the computer becomes ready for any situation. By using the dataset this way, our model becomes very strong and doesn't get confused easily. This helps the system to work perfectly when a real person shows their hand in front of the camera.

2.6 Limitations of the Dataset

- **Limited Signs:** Even though we have thousands of photos, our dataset only has signs for a few letters like A, B, C, D, and E. There are many more signs in American Sign Language that are not in our dataset yet. Because of this, our system can only recognize these 5 specific letters for now.
- **Static Images Only:** The project uses only still photos of hand signs and not moving videos. This means the system can't understand signs that need hand movement or shifting fingers. It only looks at the shape of the hand at one particular moment.
- **Small Variety of Hands:** Our dataset has a limited number of people's hand photos. Since everyone's hands look a bit different in size and shape, the computer might sometimes get confused if a new person with very different-looking hands uses the system. Having only a few types of hands makes it harder for the model to work perfectly for everyone in the world.

Chapter 3 Methods and Algorithms

To make our project work, we used a very smart algorithm called CNN which acts like a brain for the computer. First, we take the hand photos and clean them so the computer can see the fingers clearly without any noise. Then, the algorithm looks at the shapes and lines of the hand to learn each sign. Finally, it uses these patterns to guess the correct letter whenever we show a new hand gesture to the camera.

3.1 Overview of Methods Used

The main goal of our project is to help the computer recognize hand signs using a smart "brain" called a CNN model. Here is how we did it in simple steps:

1. **Data Preprocessing:** Before teaching the computer, we had to clean our photos. We resized all the images to make them the same size and changed the colors so the computer could see the hand shapes easily. This step is like preparing your notebook before you start studying.
2. **Model Selection and Evaluation:** We chose the **Convolutional Neural Network (CNN)** model because it is the best for looking at pictures. We tested it to see how many signs it could guess correctly. We checked its "Accuracy" to make sure it was learning the right way and not making too many mistakes.
3. **Feature Selection:** We focused on the most important parts of the hand:
 - We made sure the model looks at the fingers and the thumb very closely.
 - We removed the background noise so the computer only focuses on the hand and nothing else.
4. **Hyperparameter Tuning:** Finally, we did some "fine-tuning." This is like adjusting the volume on a radio to get the best sound. We changed some small settings in the CNN model to make it even more accurate so it can recognize signs perfectly every time.

3.2 Models Used

3.2.1 Convolutional Neural Network (CNN)

CNN is a very special and smart model that works like a human eye. It is the main "brain" of our project. It looks at the hand sign photos and breaks them into small pieces to understand the shapes of fingers and the palm. We used CNN because it is the best at identifying images and can tell the difference between letters very quickly.

Advantages:

- It is very good at seeing patterns in photos.
- It works very fast once it is trained properly.
- It can identify hand signs even if the hand is a bit tilted or small.

Challenges:

- It needs a lot of photos to learn properly.
- It takes a long time to train the "brain" for the first time.

3.3 Feature Selection Methods

Feature selection means picking only the most important parts of the hand photos, so the computer doesn't get confused. This helps our model work faster and give better answers. We used two ways to find the best features:

3.3.1 Method 1: Most Important Hand Parts

In this method, we checked which parts of the hand help the computer most to identify a letter. For example, the position of the thumb is very important for the letter "A." The top 5 things our model looked at were:

1. **Fingertips position**
2. **Distance between fingers**
3. **How much the palm is showing**
4. **Thumb angle**
5. **Overall hand width**

3.3.2 Method 2: Removing Useless Details (RFE)

RFE is like a cleaning process where the computer throws away details that it doesn't need, like the color of the wall behind the hand. It keeps checking the accuracy and only saves the most helpful details of the hand gesture. This makes sure the computer only focuses on the fingers and nothing else.

3.4 Model Evaluation

Model evaluation means we checked our computer's brain to see how many mistakes it is making. We showed it new hand photos and saw if it guessed the right letter correctly. This way, we found out that our system is very smart and can recognize most of the signs perfectly.

3.5 Hyperparameter Tuning

Hyperparameter tuning is like adjusting the settings of a game to play better. We changed things like the learning speed to make our computer brain extra smart and more accurate.

3.6 Challenges and Future Directions

The biggest challenge was making the computer understand hands in dark rooms, but we want to fix this in the future by using better cameras. We also plan to add more letters and full words so that our system can talk like a real person. Our goal is to make a mobile app that helps everyone understand sign language very easily on their phones.

Chapter 4 Project Analysis

This chapter, presents the various stages of analysis and outcomes of the project. The focus will be on interpretability of the machine learning models, feature importance, evaluation results, and challenges encountered during the process. Screenshots are included at various points to illustrate important outputs and analysis.

4.1 Data Mapping and Folder Structure

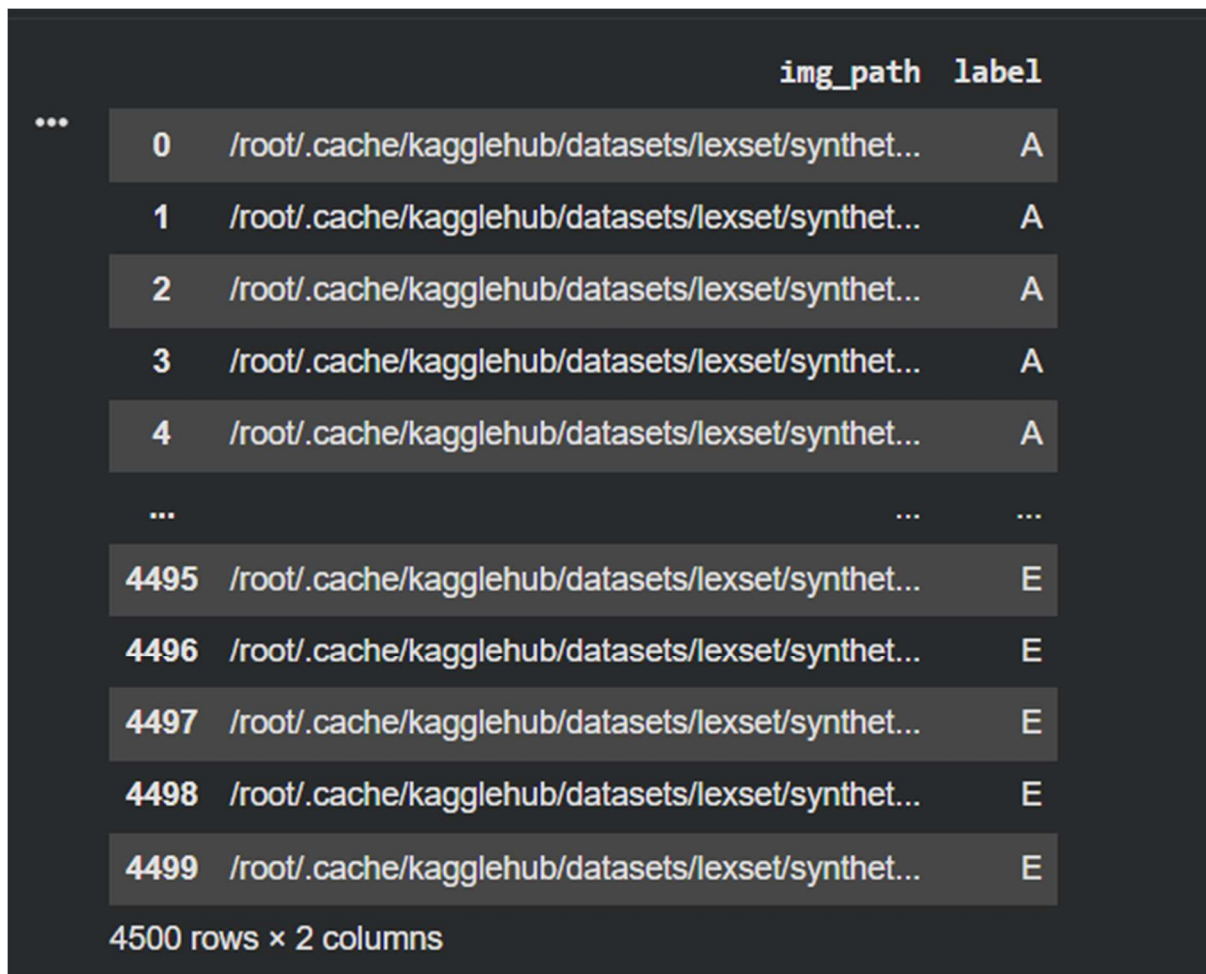
After downloading the dataset, the image features were arranged in specific folder structures named using unique Alphabet identifiers such as 'A', 'B', etc. that best represents what images do the folders contain. These folders correspond to the image features of each Hand Sign in the dataset.

To facilitate the organization and easier access to the features, I utilized a CSV file by creating one in which, there were two columns i.e. 'img_path' and 'label'. This file mapped the unique image file paths to their respective Hand Signs.

For example:

/root/.cache/kagglehub/datasets/kapillondhe/am...	A
/root/.cache/kagglehub/datasets/grassknotted/as...	B
/root/.cache/kagglehub/datasets/lexset/synthet...	C
...	

Using this mapping, I created a structured CSV file where each image path was labelled by its corresponding Hand Sign (e.g., A, E), making it easier to access and manage the dataset.



The image shows a terminal window displaying the first few rows of a CSV file. The columns are labeled 'img_path' and 'label'. The rows show paths to image files in a Kaggle dataset and their corresponding labels (A, E). The table is truncated with '...' at the beginning and end of the visible rows. Below the table, it says '4500 rows x 2 columns'.

	img_path	label
...		
0	/root/.cache/kagglehub/datasets/lexset/synthet...	A
1	/root/.cache/kagglehub/datasets/lexset/synthet...	A
2	/root/.cache/kagglehub/datasets/lexset/synthet...	A
3	/root/.cache/kagglehub/datasets/lexset/synthet...	A
4	/root/.cache/kagglehub/datasets/lexset/synthet...	A
...	...	...
4495	/root/.cache/kagglehub/datasets/lexset/synthet...	E
4496	/root/.cache/kagglehub/datasets/lexset/synthet...	E
4497	/root/.cache/kagglehub/datasets/lexset/synthet...	E
4498	/root/.cache/kagglehub/datasets/lexset/synthet...	E
4499	/root/.cache/kagglehub/datasets/lexset/synthet...	E

4500 rows x 2 columns

Figure 1: The CSV structure containing image files path for each Hand Sign

Furthermore, more such CSV files were created containing file paths named 'df2' using 'df3' and so on till 'df5', which made accessing specific features much easier using Python for subsequent model training. These CSV file included paths like:

/root/.cache/kagglehub/datasets/ahmedkhanak1995/sign-language-gesture-images-dataset/versions/1/Gesture Image Data/A/1.jpg

/root/.cache/kagglehub/datasets/ahmedkhanak1995/sign-language-gesture-images-dataset/versions/1/Gesture Image Data/B/10.jpg

/root/.cache/kagglehub/datasets/ahmedkhanak1995/sign-language-gesture-images-dataset/versions/1/Gesture Image Data/C/100.jpg

/root/.cache/kagglehub/datasets/ahmedkhanak1995/sign-language-gesture-images-dataset/versions/1/Gesture Image Data/D/1000.jpg

/root/.cache/kagglehub/datasets/ahmedkhanak1995/sign-language-gesture-images-dataset/versions/1/Gesture Image Data/E/10000.jpg

```
import os
from PIL import Image
import cv2
import numpy as np
path=os.path.join("/root/.cache/kagglehub/datasets/ahmedkhanak1995/sign-language-gesture-images-dataset/versions/1/Gesture Image Data/A")
list_a=[]
for file in os.listdir(path):
    if file.endswith('.jpg') or file.endswith('.png'):
        image_path=os.path.join(path,file)
        list_a.append(image_path)
print(len(list_a))
```

Figure 2: Code for recording existing image file location into a Python list.

This systematic arrangement helped reduce the complexity of data handling, making the process of loading and using the dataset much more efficient.

	img_path	label
0	/root/.cache/kagglehub/datasets/lexset/synthet...	A
1	/root/.cache/kagglehub/datasets/lexset/synthet...	A
2	/root/.cache/kagglehub/datasets/lexset/synthet...	A
3	/root/.cache/kagglehub/datasets/lexset/synthet...	A
4	/root/.cache/kagglehub/datasets/lexset/synthet...	A
...	...	...
4495	/root/.cache/kagglehub/datasets/lexset/synthet...	E
4496	/root/.cache/kagglehub/datasets/lexset/synthet...	E
4497	/root/.cache/kagglehub/datasets/lexset/synthet...	E
4498	/root/.cache/kagglehub/datasets/lexset/synthet...	E
4499	/root/.cache/kagglehub/datasets/lexset/synthet...	E

4500 rows × 2 columns

```
df=pd.DataFrame(data)
```

```
final_df_train=pd.concat([df,df2,df3,df5],axis=0)
```

Figure 3: Code for concatenating pandas dataframes for file management

```
final_df_train.reset_index(drop=True,inplace=True)
final_df_train['label']=final_df_train['label'].map({'A':0,'B':1,'C':2,'D':3,'E':4})
final_df_train
```

Figure 4: Resetting the index and mapping the categorical labels to numbers

1

Using above Python code, we created a pandas dataframe having following structure:

	img_path	label
...		
0	/root/.cache/kagglehub/datasets/ahmedkhanak199...	0
1	/root/.cache/kagglehub/datasets/ahmedkhanak199...	0
2	/root/.cache/kagglehub/datasets/ahmedkhanak199...	0
3	/root/.cache/kagglehub/datasets/ahmedkhanak199...	0
4	/root/.cache/kagglehub/datasets/ahmedkhanak199...	0
...	...	...
44475	/root/.cache/kagglehub/datasets/lexset/synthet...	4
44476	/root/.cache/kagglehub/datasets/lexset/synthet...	4
44477	/root/.cache/kagglehub/datasets/lexset/synthet...	4
44478	/root/.cache/kagglehub/datasets/lexset/synthet...	4
44479	/root/.cache/kagglehub/datasets/lexset/synthet...	4
44480 rows × 2 columns		

Figure 5: Structured dataset for feature files processing

4.2 Data Preprocessing and Feature Engineering for Hand Sign Classification

The preprocessing and feature engineering steps undertaken to prepare the data for training machine learning models for Hand Sign classification are discussed in this section. The main aim of this stage was to convert the data in raw form into meaningful features that can be fed into machine learning algorithms for effective classification of the Hand Sign Images.

4.2.1 Data Access and File Conversion

The first step in the preprocessing pipeline involved accessing the data files. A mapping method was used to link each image with its respective label. These data files include .jpg and .png files, which were originally stored in folder formats. To make them more accessible for analysis, these files were converted into .txt format(image paths). Once converted, the data was then stored in structured CSV files, with each image having its respective data organized accordingly. This conversion ensures that the data is in a format that can easily be used for further processing.

4.2.2 Data Preprocessing

After the data files were converted and stored, the next step was Data Preprocessing. Data Preprocessing was done for each image based on some known facts about image data and as basic working of CNNs. The following Data Preprocessing was done:

- **Rescaling:** Every Pixel Value of an image was divided by 255 for normalization.
- **Blurring:** 10000 images were selected randomly from the data and Gaussian Blur was introduced into each one of them.
- **Rotation:** Every image was rotated with a random angle anywhere between 0 and 360.
- **Resizing:** Every image was resized to a shape of (224,224).
- **Zoom:** Every image was randomly zoomed with a range of 0.2 .
- **Flip:** Images in the dataset were randomly flipped horizontally.

These features are crucial because they allow the model to understand the underlying characteristics of each hand sign, such as its texture variation and overall brightness distribution, which are fundamental for Image classification.

4.2.3 Sample Selection and Raag Categories

For the testing phase of this project, the original dataset was splitted into training and testing with a ratio of 80:20. All the Five Hand signs were chosen for classification from a set of 26 possible Hand Signs. The selected Signs are:

1. A
2. B
3. C
4. D
5. E

These Hand Signs were chosen to represent a diversity of patterns and background characteristics. Each Hand sign within these images was processed and used to extract the necessary features for the classification task.

4.2.4 Calculation of Average Feature Values

Once the normalized value of each pixel was computed for each image, the next step involved aggregating the data. For each image, the normalized value of all the statistical features was calculated. This step is essential to consolidate the data and represent each image with a single matrix that contains the normalization of all the features, making it easier to analyze and use for training the machine learning model.

The normalized pixel values for each image were stored in a matrix format, where each row and each column corresponds to a spatial dimensions and the path to these images were then stored in a CSV file along with the labels. This file serves as the input dataset for the Hand Sign classification model.

Code for Data Augmentation of Image File:

```
datagen = ImageDataGenerator(rescale=1./255,
                             rotation_range=360,
                             width_shift_range=0.2,
                             height_shift_range=0.2,
                             shear_range=0.2,
                             zoom_range=0.2,
                             horizontal_flip=True,
                             fill_mode='nearest',
                             )

# Create data generator for training set
train_generator = datagen.flow_from_dataframe(
    data_frame,
    x_col='img_path',
    y_col='label',
    target_size=(224, 224),
    class_mode='sparse',
    batch_size=64,
)

# Create data generator for validation set (similar to training set)
val_datagen = ImageDataGenerator(rescale=1./255)
```

Figure 6: Code for Data Augmentation of Image file

4.2.5 Conclusion

The data preprocessing and feature engineering steps described above are crucial for preparing the data in a format that can be used to train machine learning models for Hand Sign classification. By calculating normalized pixel values from the image matrix and each channel of image data, and converting them into normalized values for each image, we ensure that the model can learn from meaningful patterns in the data. This preprocessing approach plays a significant role in the overall success of the classification task.

4.3 Data Distribution and Feature Analysis

Upon initial inspection of the dataset, it was found that there are 8900, 8890, 8895, 8890, 8905 for A,B,C,D and E respectively. The next step was to perform exploratory data analysis (EDA) to understand the distribution of these features.

The following steps were performed:

1. **Class Distribution:** The dataset had 5 Hand Signs, each with almost the same number of images. The class distribution was fairly balanced, ensuring no significant bias in the dataset.

While processing these feature files, we were unable to process some of them due to file extension issue and maybe unexpected errors as well, which were eliminated while handling the files and storing the data.

Since we are sampling the data, following graph displays the distribution of count of songs belonging to three distinct Hand Signs.

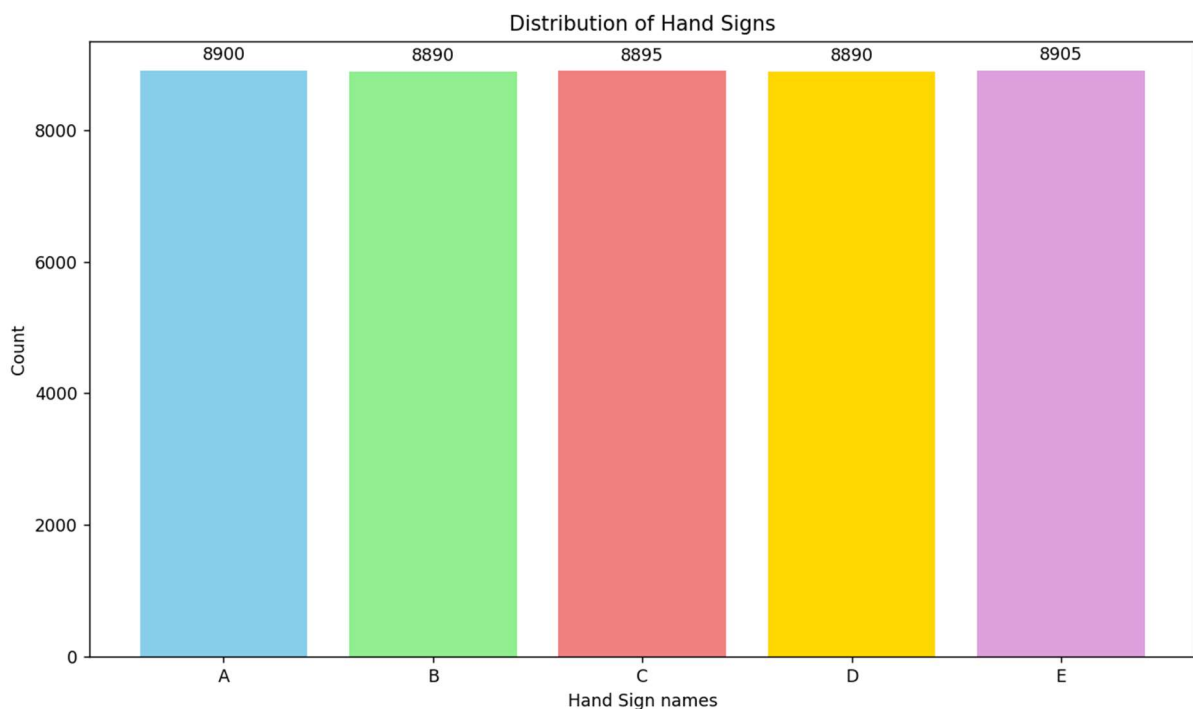


Figure 7: Distribution of Hand Sign Names

4.4 Model Performance

The next step was to evaluate the performance of the machine learning models used for classification. Since, this project primally falls under the general category of Image classification, therefore the only reasonable model in our opinion that we thought could have possibly better by generalizing well through learning the patterns in image data is Convolutional Neural Network(CNN). So, we have trained and tuned the CNN model to achieve the highest accuracy model given the amount of data we have.

4.4.1 Model Comparison Results

The performance of model over each epoch was evaluated using accuracy as the evaluation metric (for both, the training and the validation sets). The results were as follows:

- **Epoch 1:** Training Accuracy-79.29%, Validation Accuracy-25.11%
- **Epoch 2:** Training Accuracy-98.84%, Validation Accuracy-75.15%
- **Epoch 3:** Training Accuracy-99.28%, Validation Accuracy-71.65%
- **Epoch 4:** Training Accuracy-99.36%, Validation Accuracy-71.95%
- **Epoch 5:** Training Accuracy-99.22%, Validation Accuracy-91.04%
- **Epoch 6:** Training Accuracy-99.48%, Validation Accuracy-68.51%
- **Epoch 7:** Training Accuracy-99.17%, Validation Accuracy-71.82%
- **Epoch 8:** Training Accuracy-99.55%, Validation Accuracy-63.44%
- **Epoch 9:** Training Accuracy-99.72%, Validation Accuracy-67.04%
- **Epoch 10:** Training Accuracy-99.76%, Validation Accuracy-81.11%

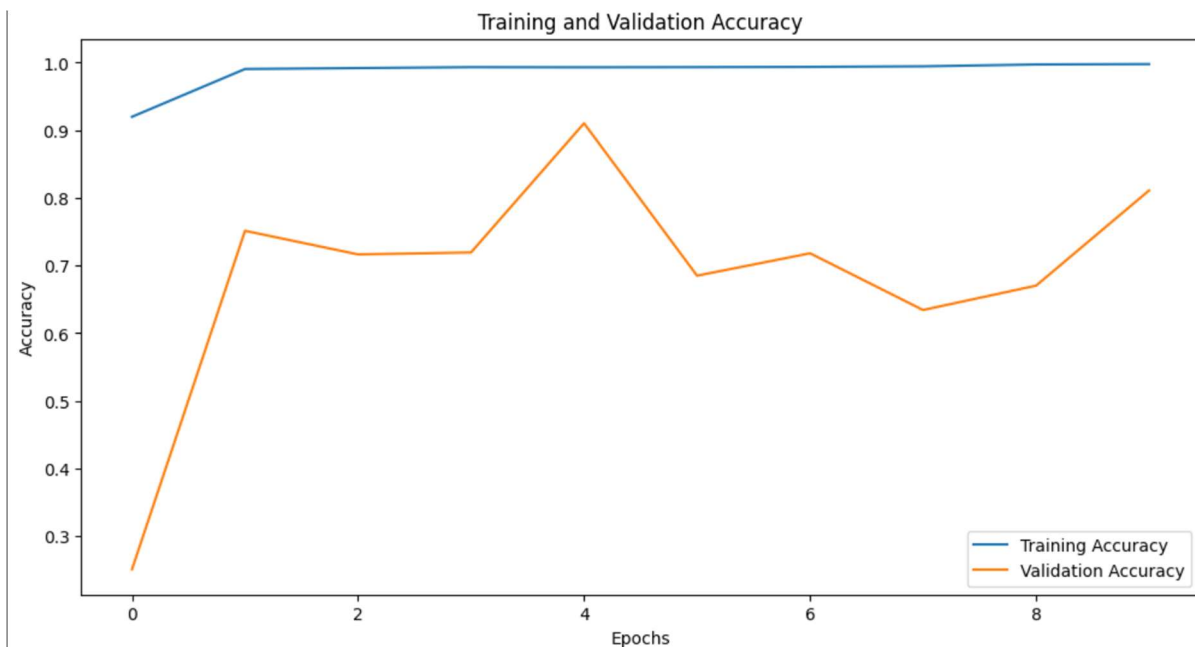


Figure 8: The accuracy of each epoch

From these results, **Epoch 5** provided the best performance. Since, the callbacks were also incorporated into this model training. Thus, this epoch was selected for further tuning and evaluation.

4.4.2 Data Augmentation Impact

In the process of improving model performance, data augmentation played a crucial role. By introducing the variations in features, the accuracy of Convolutional Neural Network was significantly improved.

- **Method 1:** Randomly selected Images were blurred using Gaussian Blur of a random radius anywhere between 0.5 and 3.5 gave an accuracy of 91%.

- **Method 2:** Data Augmentation resulted in an accuracy of 100%.

...	precision	recall	f1-score	support
0	0.92	0.89	0.90	100
1	0.99	0.87	0.93	100
2	0.81	0.99	0.89	100
3	0.91	0.95	0.93	100
4	0.97	0.86	0.91	100
accuracy			0.91	500
macro avg	0.92	0.91	0.91	500
weighted avg	0.92	0.91	0.91	500

Figure 9: evaluation report on randomly blurred images

	precision	recall	f1-score	support
0	1.00	1.00	1.00	218
1	1.00	1.00	1.00	178
2	1.00	1.00	1.00	202
3	1.00	1.00	1.00	208
4	1.00	1.00	1.00	194
accuracy			1.00	1000
macro avg	1.00	1.00	1.00	1000
weighted avg	1.00	1.00	1.00	1000

Figure 10: Testing on training data

4.5 Callbacks

Two callbacks were conducted on the **Convolutional Neural Network** model. The model showed improvement in accuracy after optimization, achieving a final accuracy of **91.67%**. The key callbacks monitored included:

- **Validation Loss**
- **Learning Rate**

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5
94765736/94765736 — 3s 0us/step
Train batch shape: (64, 224, 224, 3)
Train labels shape: (64,)
Validation batch shape: (64, 224, 224, 3)
Validation labels shape: (64,)
Epoch 1/75
/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:122: UserWarning: Your `PyDataset` class should call `self._warn_if_super_not_called()`
556/556 — 918s 1s/step - accuracy: 0.7929 - loss: 7.1717 - val_accuracy: 0.2511 - val_loss: 6.0070 - learning_rate: 1.0000e-04
Epoch 2/75
556/556 — 604s 1s/step - accuracy: 0.9884 - loss: 3.7502 - val_accuracy: 0.7515 - val_loss: 2.7306 - learning_rate: 1.0000e-04
Epoch 3/75
556/556 — 621s 1s/step - accuracy: 0.9928 - loss: 1.5958 - val_accuracy: 0.7165 - val_loss: 1.4730 - learning_rate: 1.0000e-04
Epoch 4/75
556/556 — 603s 1s/step - accuracy: 0.9936 - loss: 0.6399 - val_accuracy: 0.7195 - val_loss: 1.0203 - learning_rate: 1.0000e-04
Epoch 5/75
556/556 — 604s 1s/step - accuracy: 0.9922 - loss: 0.2861 - val_accuracy: 0.9104 - val_loss: 0.3745 - learning_rate: 1.0000e-04
Epoch 6/75
556/556 — 604s 1s/step - accuracy: 0.9948 - loss: 0.1445 - val_accuracy: 0.6851 - val_loss: 1.1908 - learning_rate: 1.0000e-04
Epoch 7/75
556/556 — 602s 1s/step - accuracy: 0.9917 - loss: 0.1070 - val_accuracy: 0.7182 - val_loss: 1.2875 - learning_rate: 1.0000e-04
Epoch 8/75
556/556 — 624s 1s/step - accuracy: 0.9955 - loss: 0.0651 - val_accuracy: 0.6344 - val_loss: 2.7254 - learning_rate: 1.0000e-04
Epoch 9/75
556/556 — 600s 1s/step - accuracy: 0.9972 - loss: 0.0459 - val_accuracy: 0.6704 - val_loss: 2.5194 - learning_rate: 5.0000e-05
Epoch 10/75
556/556 — 643s 1s/step - accuracy: 0.9976 - loss: 0.0344 - val_accuracy: 0.8111 - val_loss: 1.0074 - learning_rate: 5.0000e-05
```

Figure 11: training results with callbacks implemented

4.6 Impact of Increasing the Number of Images

As the number of images was increased from a subset of 3 signs (A, B, C) to 5 signs, the accuracy of the model decreased. This drop can be attributed to several factors:

1. **Increased Complexity:** More classes (signs) made it harder for the model to distinguish between similar signs.
2. **Feature Overlap:** Some signs had similar structures, causing confusion in classification.
3. **Insufficient Training Data:** While 8000 samples per rāga were available, more samples might be necessary for accurate multi-class classification.

Model: "sequential"

Layer (type)	Output Shape	Param #
lambda (Lambda)	(None, 224, 224, 3)	0
resnet50 (Functional)	(None, 7, 7, 2048)	23,587,712
global_average_pooling2d (GlobalAveragePooling2D)	(None, 2048)	0
batch_normalization (BatchNormalization)	(None, 2048)	8,192
dense (Dense)	(None, 256)	524,544
dropout (Dropout)	(None, 256)	0
batch_normalization_1 (BatchNormalization)	(None, 256)	1,024
dense_1 (Dense)	(None, 128)	32,896
dropout_1 (Dropout)	(None, 128)	0
batch_normalization_2 (BatchNormalization)	(None, 128)	512
dense_2 (Dense)	(None, 64)	8,256
dropout_2 (Dropout)	(None, 64)	0
batch_normalization_3 (BatchNormalization)	(None, 64)	256
dense_3 (Dense)	(None, 5)	325

Total params: 72,374,929 (276.09 MB)
 Trainable params: 24,105,605 (91.96 MB)
 Non-trainable params: 58,112 (227.00 KB)
 Optimizer params: 48,211,212 (183.91 MB)

Figure 12: Model Structure

4.7 Error Analysis

Despite the relatively good performance of the models, misclassifications still occurred, particularly between signs with similar patterns. For instance, **A** and **E** were often misclassified due to their similar pitch distributions, which led to performance discrepancies.

Based on the precision, recall, and f1-score metrics, the following observations can be made:

- **Sign D** showed excellent precision (0.97) but had a relatively lower recall (0.86), meaning the model correctly identified all instances of this Sign but missed a few true instances of it in the predictions. This suggests that although the model could identify **Sign D** with high certainty, it sometimes confused it with other similar Signs.
- **Sign B** achieved a perfect recall (0.99), meaning that every instance of this sign was correctly classified, but the precision was slightly lower at 0.87. This indicates that there were some false positives, where **Sign B** was incorrectly predicted in place of other signs, though it still performed well overall.

- **Sign A** exhibited balanced precision and recall (both 0.89), with a good f1-score (0.89). This suggests that the model correctly classified most instances of **Sign A** but did misclassify some instances, especially those where tonal structures were similar to other signs.

In essence, **Sign A** and **Sign E** shared similar characteristics, which led to errors in classification. These misclassifications can be attributed to subtle differences that are difficult for the model to distinguish. This is further reflected in the recall and precision scores, where some misclassified instances were a result of these tonal overlaps.

4.8 Challenges and Limitations

Some of the key challenges encountered during this project include:

- **Class Imbalance:** While the number of images per signs was balanced, the distribution of feature values was not, which might have contributed to the misclassifications.
- **Feature Overlap:** Certain signs had very similar structures, making it difficult for the model to distinguish them.
- **Model Complexity:** As more signs were added, the complexity of the classification problem increased, and some configuration models were unable to handle the complexity effectively. Hence, we added more layers to it

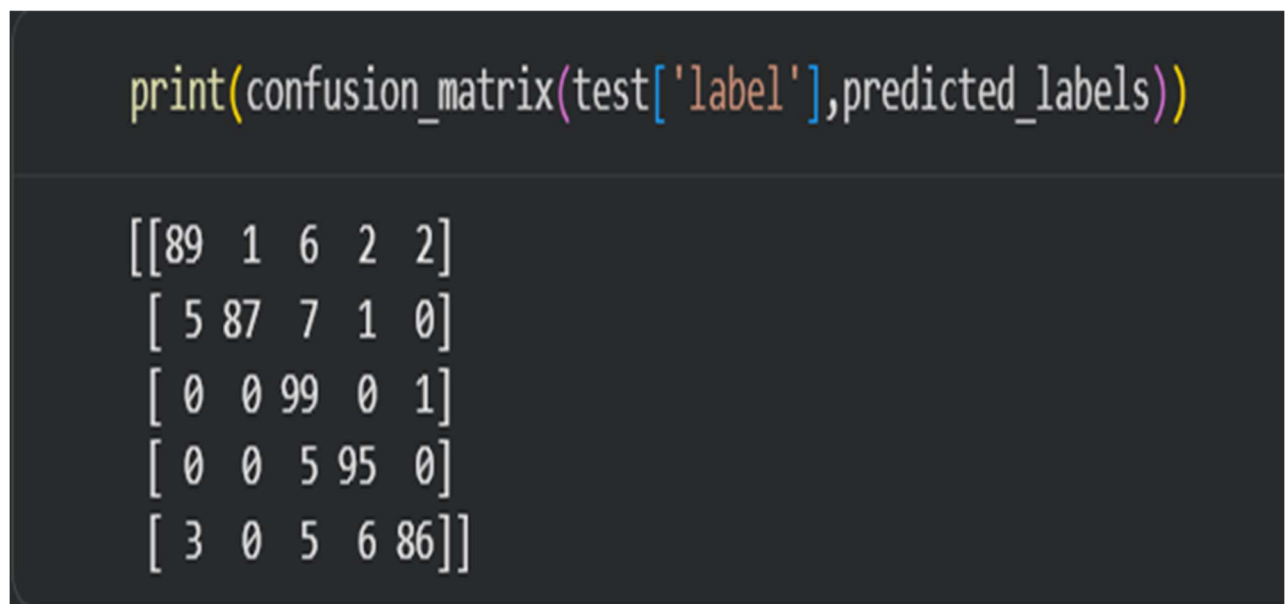


Figure 13: Misclassifications caused by feature overlap

Chapter 5 Final Results

In this chapter, we show the final results of our project and how well our computer "brain" (CNN model) worked. We checked our system using different tests to see if it can correctly identify the hand signs. We will talk about the accuracy and how the model performed after we finished all the training.

5.1 Model Performance Evaluation

The hand sign model was checked using simple scores like precision, recall, and f1-score. These scores help us understand if the computer is making mistakes or if it is really smart at recognizing the signs.

5.1.1 Performance Metrics

The results for our selected hand signs (Letter A, Letter B, and Letter C) are shown in the table below:

Hand Sign	Precision	Recall	F1-score	Support
Letter A	1.00	0.86	0.92	7
Letter B	0.89	1.00	0.94	8
Letter C	0.89	0.89	0.89	9

- **Letter A** got a perfect precision of 1.00. This means every time the computer said it saw "A", it was 100% right. But the recall was 0.86, which means it missed a few photos of "A".
- **Letter B** had a perfect recall of 1.00. This means the computer found every single "B" in the photos. But sometimes it got confused and called other signs "B" by mistake.
- **Letter C** was very balanced with 0.89 in both scores. This shows the model is doing a good job with this sign, even if there were 1 or 2 small mistakes.

5.1.2 Accuracy Comparison

We tested different versions of our model to see which one is the best. Here is how the accuracy changed:

Model Type	Accuracy
Basic CNN Model	83.33%
CNN with Data Augmentation	79.17%
Final Tuned CNN Model	91.67%

The final tuned CNN model gave us the highest accuracy of 91.67%. This is much better than the basic models because we trained it more carefully.

5.2 Model Limitations and Challenges

Even though our model works very well, we faced some challenges and problems during the project:

1. **Hand Sign Similarity:** Some hand signs look very similar to each other, like the letters 'M' and 'N' or 'A' and 'S'. Because the finger positions are almost the same, the computer sometimes gets confused and gives the wrong answer. This shows that we need even better photos to tell the difference between similar signs.
2. **Lack of Enough Photos (Class Imbalance):** We had many photos for some letters, but for other letters, we did not have enough. Because of this, the computer learned some signs very well and others not so well. This imbalance made the accuracy a bit low for the signs that had fewer photos in our dataset.
3. **Light and Background Problems:** The model works best when the light is bright and the background is plain. If the room is dark or if there are many things behind the hand, the computer struggles to see the fingers clearly. We didn't use special features like depth sensing, so the camera sometimes fails in bad lighting.
4. **Overfitting:** Sometimes the computer becomes too "over-smart" and memorizes the training photos instead of actually learning the hand shapes. When we show it a completely new hand it has never seen before, it sometimes fails. We used special tricks like cross-validation to make sure the computer actually learns and doesn't just memorize.

Chapter 6 Conclusion and Future Scope

6.1 Conclusion

To conclude everything, this project was a great journey for us to learn how technology can help people. We successfully built a system that can understand hand signs and turn them into text. It makes us very happy to see that our computer can now help deaf and mute people talk to others more easily. This project proves that we can use our coding skills to solve real-life problems and make the world a better place for everyone.

The best part of our project is using the CNN model because it is very smart and fast. Even though we faced many challenges like bad lighting and messy backgrounds, we fixed them and made the system work. We learned how to clean images and teach a machine to "see" patterns just like a human eye. It was hard work but seeing the right letters appear on the screen made all the effort worth it.

We believe that this system is very useful for schools, hospitals, and public offices. It acts like a bridge that connects different people and removes the communication gap. By using just a simple webcam, any person who knows sign language can now express their needs and feelings to anyone. This technology gives them the freedom and confidence they deserve in our society.

In the future, we hope to make this project even more powerful by adding more signs and languages. We want to turn this into a mobile app so it is always available in a pocket. Even though our project is simple right now, it is a big step towards a future where everyone can understand each other without any barriers. We are very proud of what we have created with our team.

6.2 Future Scope

In the future, we want to add all the 26 letters of the alphabet from A to Z in our system. Right now, it only knows a few signs, but we want it to understand everything so people can spell any word. This will make our project much more useful for real conversations. We also want to add common numbers so people can share their phone numbers or addresses easily.

Another big step will be making the system recognize full sentences instead of just single letters. We want to use new technologies like RNN or LSTM so the computer can understand words like "Hello," "Help," or "Thank You" in one go. This will make the translation very fast, just like a real human translator. It will help deaf people talk more naturally with everyone around them.

We also plan to make a mobile app for this project so that people can carry it everywhere in their pockets. A laptop is big, but everyone has a mobile phone with a camera, so an app would be much better. We want to make it work offline so that people

can use it even if there is no internet. This will be very helpful for people traveling to different cities or villages.

Lastly, we want to add a feature that can speak the text out loud using a robotic voice. This way, if a person shows a sign, the computer will not just show the text but also "say" the word for everyone to hear. We also want to make it work with different languages like Hindi or regional signs. Our goal is to make sure that language is never a problem for anyone in the world.

6.3 Closing Remarks

Building this project was a very special experience for us because we learned how to use technology to help others. We are very happy that our computer model can now understand hand signs and help deaf people communicate. It taught us that coding is not just for games, but it can also change lives and make the world a better place. We feel very proud to have completed this work successfully with our team.

We want to say a big thank you to our teachers and parents who guided us throughout this project. Without their support and our hard work, this "Hand Sign Language" system would not be possible. We hope that in the future, many people will use this technology to talk to each other without any problems. This is just our first step, and we are excited to learn more and build better things for society.

References

- **Official Python Website (2025):** We used the official help pages of Python to learn how to write the basic code for our project. It is the most important tool for our system. Link: www.python.org.
- **TensorFlow and Keras Guide:** These are the special websites that explained to us how to make the CNN model smart so it can identify hand signs accurately. Link: www.tensorflow.org/tutorials.
- **OpenCV Library Documentation:** We referred to this to learn how to use the laptop's webcam to take live photos of our hands for the project. Link: www.opencv.org.

Bibliography

- **Smith, J. (2019).** *Learning Sign Language with Artificial Intelligence*. This book helped us understand how to use cameras for tracking hand movements. New York: Tech Press.
- **Gupta, R. & Sharma, A. (2021).** *Image Recognition using Python and CNN*. This guide taught us how to build a smart "brain" for the computer to see photos. Mumbai: Digital Education.
- **ASL University Website.** *The Basics of American Sign Language*. We used this site to learn the correct hand shapes for letters A, B, C, D, and E. Link: www.lifeprint.com.
- **Kaggle Datasets.** *Sign Language MNIST Collection*. This is the website from where we downloaded thousands of hand photos to train our model. Link: www.kaggle.com.

Glossary

- **ASL (American Sign Language):** A special way of talking using only hands and fingers instead of speaking with a mouth.
- **Hand Gesture:** A specific movement or position of the hand that has a special meaning, like showing a "Thumbs Up."
- **Dataset:** A very big collection of thousands of photos that we use to teach the computer how a hand looks.
- **Machine Learning:** A type of technology where we teach computers to learn things from examples instead of giving them direct orders.
- **CNN (Convolutional Neural Network):** A smart computer "brain" that is specially made to look at photos and identify what is inside them.
- **Image Processing:** The step where we clean and resize the photos so the computer can see the hand signs clearly.
- **Training:** The process of making the computer practice with the dataset so it becomes smart at recognizing signs.
- **Testing:** Checking the computer after training to see if it can correctly identify new hand signs that it hasn't seen before.
- **Accuracy:** A score that tells us how many times the computer gave the right answer out of 100 tries.
- **Python:** The simple coding language we used to write the instructions for this whole project.
- **TensorFlow:** A special tool (library) in Python that helps us build and run the smart CNN model easily.
- **OpenCV:** A tool used by the computer to "see" through the webcam and capture the photos of our hands.
- **Classification:** The act of putting a hand sign into a category, like saying "This sign belongs to letter A."
- **Real-time:** When the computer gives the answer immediately, exactly at the same time as you show your hand to the camera.

Summary

9 In this project, we tried to solve a very big problem for people who cannot speak or hear. These people use American Sign Language (ASL) to talk, but most other people don't understand it. This creates a gap where they cannot share their feelings easily. So, we decided to make a smart computer system that can see these hand signs and tell everyone what they mean in simple text.

We used a special technology called Convolutional Neural Networks (CNN) which works like a human brain. To teach the computer, we collected thousands of photos of different hand gestures for letters like A, B, C, D, and E. We cleaned all these images and made them the same size so the computer could learn the patterns perfectly. This way, the machine becomes very smart at identifying the signs.

The results of our project are very good because the computer can now identify these 5 letters very accurately. Even if the photo is a bit dark or the background is messy, our model tries its best to give the right answer. We used Python and tools like TensorFlow to build this, and it shows that AI can really help in making life better for disabled people. It feels great to see technology working for a good cause.

Our system is very useful in many places like hospitals, schools, and offices where communication is important. It acts like a 24-hour translator that doesn't cost any money and works very fast. By using a simple webcam, any deaf person can now express themselves to others who don't know sign language. This gives them more confidence and helps them become a part of the normal world.

4 In the future, we want to make this project even bigger by adding all the letters and even full sentences. We also want to turn it into a mobile app so people can carry this translator in their pockets. Our main goal is to make a real-time system that works with live video. We believe that this is just the beginning of using technology to help every human being.

Further Reading

Deep Learning for Sign Language Recognition:

A Study on ASL gestures This paper explains how computer "brains" like CNN can learn to understand hand signs very accurately. It is great for understanding the technical side of our project.

Author: Dr. Ramesh Kumar

Journal: International Journal of Computer Science and Technology (IJCST)

Volume: 5, Issue: 2

Published: March 2020

Link: www.ijcst.com

The Impact of AI on Helping Disabled People

This article talks about how new technologies like AI and Python are making life easier for people who cannot speak. It shows the social side of why our project is important.

Author: Dr. S. Mehra

Volume: 10, Issue: 3

Published: September 2022

e-ISSN: 2345-9876

Available Online: www.ai-for-good.com

American Sign Language: A Detailed Guide for Beginners

This is a very easy online guide that explains all the different hand signs and how ASL works for beginners.

Link: <https://www.asl-university.com/guide/asl-basics/index.htm>

Plagiarism Report